

Performance or Efficiency? A Tale of Two Cores for DB Workloads

Rathijit Sen

Gray Systems Lab, Microsoft
Redmond, Washington, USA
rathijit.sen@microsoft.com

ABSTRACT

We study the performance, power, and thermal profiles for database workloads on hybrid P-core and E-core CPUs. We find that E-cores run cooler than P-cores, use less power, and are more energy-efficient, but need to be provisioned more than 3x the number of P-cores for equivalent performance on multi-threaded analytics workloads. For latency-critical transactional workloads, E-cores can significantly extend very short tail latencies but have less impact for workloads with longer latencies. Asymmetric work distribution policies skewed towards P-cores can improve grouped aggregation performance compared to symmetric policies.

ACM Reference Format:

Rathijit Sen. 2024. Performance or Efficiency? A Tale of Two Cores for DB Workloads. In *20th International Workshop on Data Management on New Hardware (DaMoN '24)*, June 10, 2024, Santiago, AA, Chile. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3662010.3663444>

1 INTRODUCTION

High core-count CPUs benefit well-parallelized multi-threaded workloads, but having high-performance features for all cores to benefit single-thread performance as well can exceed chip power, thermal, and area budgets [4, 15, 25]. One approach to partially mitigate this issue is using dynamic management, e.g., frequency (and voltage) scaling [2, 19, 24]. Another largely orthogonal approach is having asymmetric, i.e., heterogeneous or hybrid cores [18, 25, 28], e.g., in designs by ARM [5, 6], Intel [32], AMD [9, 10], and others.

We focus on Intel's recent hybrid architecture [32] that combines Performance cores (P-cores), targeting high single-thread performance, and Efficiency cores (E-cores), targeting high energy efficiency (performance-per-watt) and area efficiency, on the same CPU chip. Figure 1 shows an example of such a hybrid CPU, with clusters of smaller E-cores having a shared L2 cache unlike the larger and faster P-cores. Currently such hybrid designs are marketed for desktop and mobile uses with E-core:P-core (physical cores) ratios of 1:1, 1:2, 2:3, 4:3, 2:1, 4:1, etc. [43, 46]. Interestingly, high core count servers with only E-cores are on the horizon [8], giving service providers the option of choosing between P-core only or E-core only servers. Thus, for proper resource provisioning for database workloads, it becomes important to understand the power-performance profiles for these core types. Most prior works about multicore database performance analysis [29, 33, 34, 38–41] have

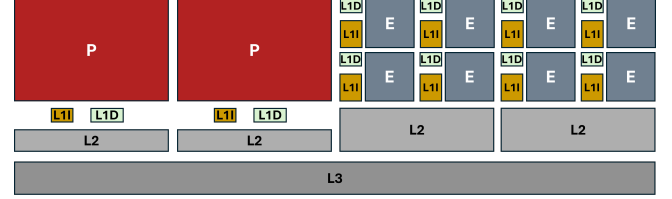


Figure 1: Cores and cache hierarchy of an example hybrid CPU (Raptor Lake). Components are not drawn to scale.

been on homogeneous CPU cores per socket, but much less [26, 42] on heterogeneous cores.

In this work, we study the following unexplored questions.

- How do performance, power, and thermal profiles for running database workloads with P-cores compare with those of E-cores?
- What E-core:P-core ratio may be needed to get similar performance between E-cores and P-cores?
- Is there a performance benefit with non-uniform/asymmetric work distribution policies on hybrid P-core, E-core CPUs?

System Under Test: Our test machine has a single-socket Intel i9-14900K (Raptor Lake) CPU [21] with 8 physical P-cores (16 logical cores with hyper-threading) and 16 E-cores (no hyper-threading), 128 GB DDR5 memory, NVMe SSD, and runs Ubuntu 22.04.3 LTS.

The microarchitectures of the P-cores and E-cores for this CPU are Raptor Cove [45] and Gracemont [44] respectively. Nominal frequencies for the P-cores and E-cores are 3.2 GHz and 2.4 GHz respectively, while maximum turbo frequencies are 5.6-6 GHz and 4.4 GHz. Two logical cores (with hyper-threading enabled) on each physical P-core share a 32 KB L1 instruction cache (L1I), a 48 KB L1 data cache (L1D), and a 2 MB L2 cache. Each E-core has a 64 KB L1I and a 32 KB L1D. Each cluster of four E-cores has a shared 4 MB L2-cache. The 36 MB L3 cache is shared by all cores. Other microarchitectural differences include a micro-op cache for P-cores, larger sizes of the reorder buffer, register files, load and store queues, ITLB, and DTLB for P-cores compared to E-cores, etc. Due to E-core limitations, currently hybrid CPUs do not support AVX-512. The maximum socket power for this CPU is 253 W. The machine (wall) power at idle is ~51 W. We use PCM [22] to access hardware counters and a power meter to monitor machine power consumption.

2 ANALYTICAL WORKLOADS

For experiments in this section, we run TPC-H queries with scale factor 100 on DuckDB [30] v0.10.0 using its Python interface.

Figure 2 shows the performance ratio for each query (top) and each relational operator (bottom, in descending order by their total times over all queries) by using only P-core threads vs using only E-cores. For queries, the speedup ranges from 8.2% (Q13)



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike International 4.0 License.

DaMoN '24, June 10, 2024, Santiago, AA, Chile
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0667-7/24/06.
<https://doi.org/10.1145/3662010.3663444>

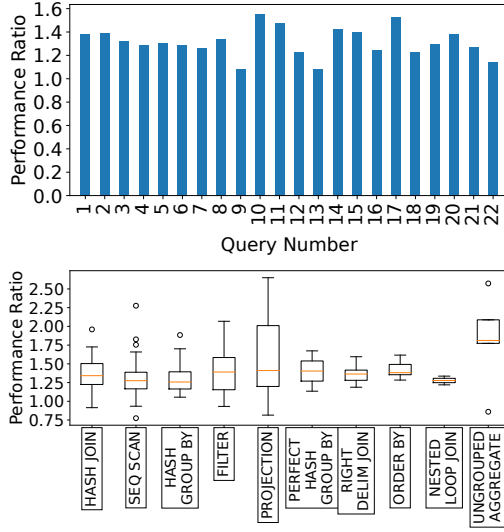


Figure 2: Speedup with 16 (logical) P-cores over 16 E-cores for each TPC-H query (top) and relational operator (bottom).

to 55.8% (Q10), with a geometric mean of 30.9% over all queries. Operator speedups are also query-dependent, with an average of 27.8% (nested-loop join) to 82.1% (ungrouped aggregate). Thus, simply scaling query or operator time for one core type by a constant factor for another core type is not accurate. Therefore, augmenting existing query performance modeling and prediction techniques [1, 12, 13, 27] with heterogeneity awareness could be useful for resource allocation and query scheduling in hybrid CPUs.

Figure 3 shows values of performance, power, and thermal metrics¹ for the workload consisting of all 22 TPC-H queries, using only E-cores, using only P-cores, and using all 32 logical cores (dashed line) in the system. We vary the number of threads (n). For execution on only P-cores, we use one logical core each on n different physical cores for $n = 2, 4, 8$, to get the best performance with n P-core threads, and all 16 logical (i.e., 8 physical) P-cores for $n = 16$.

Queries-Per-Second (QPS, Figure 3a) increases with n for both P-cores and E-cores, but scales more with physical P-cores. QPS with n P-cores is comparable ($n = 2$) to, or larger ($n > 2$) than, with $2n$ E-cores (5.4%, 10.5% larger for $n = 4, 8$ respectively). While QPS for P-cores increases by 69.9% by doubling n from 4 to 8, it increases further by only 16.6% with $n = 16$ threads, showing less benefits of hyper-threading for P-cores. QPS can be further increased by 20.2% by using the 16 E-cores as well, reaching an ALLcores QPS of 0.8.

An increase in n and QPS also results in higher average memory bandwidth used (Figure 3b), higher socket power consumption (Figure 3c), and lower thermal headroom² (Figure 3d). ALLcores uses more power (socket: 190.9 W, machine: ~295 W) than only 16 P-core threads (socket: 146.6 W, machine: ~228 W) or only 16 E-cores (socket: 102.3 W, machine: ~166 W). Performance-per-Watt

¹Error bars in all figures show ± 1 standard deviation of measured values for the metrics between multiple runs of the workload.

²Thermal headroom = max. thermal junction temperature [20] – core temperature. Higher thermal headroom lowers cooling costs and allows for frequency boosts with reduced likelihood of thermal throttling. We show the minimum headroom across all cores in the socket, averaged over the run of the workload.

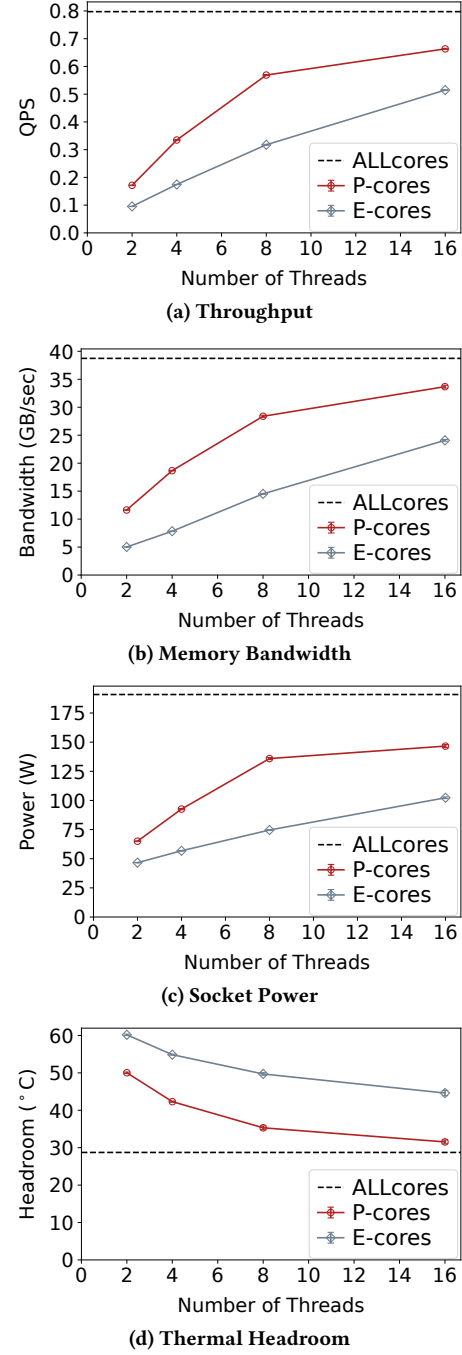


Figure 3: Avg. performance, power, and thermal headroom for TPC-H. ALLcores values are with all 32 logical cores used.

with 16 E-cores is better than for 16 P-core threads (socket: 11.2%, machine: ~7%) and for ALLcores (socket: 20.5%, machine: ~15%). E-cores also run cooler than P-cores, e.g., the thermal headroom with 8 E-cores is almost the same as that with 2 P-cores (2 threads).

To gain insights about resource provisioning, we fit Amdahl's Law model [3, 18], as a linear function of $1/n$, to the measured

values for average query time ($= 1/\text{QPS}$) and a linear regression model, as a function of n , to the measured values for average socket power. For E-cores, we use measured values for $n = 2, 4, 8$, and 16 , and see a good fit with R^2 values of greater than 0.99 . For P-cores, we only use measured values for $n = 2, 4$, and 8 to fit the models and get a similarly good fit. Thus, the models assume that each thread is on a different physical core which reflects our approach of seeking the best P-core performance with n P-core threads. Figure 4 shows model-estimated values for 1–9 P-core threads (using one logical core per physical P-core) and 1–28 E-cores, along with the measured values for 2, 4, 8, and 16 E-cores and logical P-cores. Note that the rightmost measured values for P-cores in the plot correspond to 16 logical P-cores ($= 8$ physical P-cores on this CPU) and are close to the estimated values for 9 P-core threads on 9 physical P-cores.

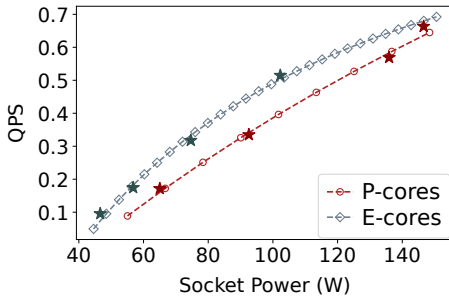


Figure 4: Estimated avg. socket power vs. QPS for 1–9 P-core threads and 1–28 E-cores. Measured values for 2, 4, 8, 16 P-core threads and E-cores on this CPU are marked with ★.

The estimates also suggest that if we use only E-cores, then we need 26 E-cores to get performance similar to using only 16 logical P-cores on this CPU, without exceeding the socket power of the latter. Thus, for resource provisioning for equivalent performance for this workload, the E-core:P-core ratio should be at least 3.25:1 for $n = 16$ ($= 8$ physical P-cores on this CPU). Since E-cores are grouped in clusters of four cores each, this may translate into 28 E-cores in practice and hence need some more area and power. The ratio will likely be greater for larger n due to the faster P-cores and diminishing returns for query performance at high thread counts, which then requires more E-cores to catch up (if possible) to P-core performance. Reduced serial overheads for query execution or concurrent query execution can reduce the E-core:P-core ratio needed for equivalent throughput at high core counts.

3 TRANSACTIONAL WORKLOADS

Here we evaluate how latency-critical workloads perform on P-cores vs on E-cores. We run (a) TPROC-C [37] (50 users, 250 warehouses) using HammerDB [36] CLI v4.9 on PostgreSQL [16] v16.1 and (b) memtier benchmark [31] (8 threads, 50 clients per thread, 100 K requests per client, key values $\in [0, 4095]$) on memcached. We chose these two workloads to study different scenarios with latencies in tens of milliseconds and in much shorter sub-millisecond ranges. We run the server threads on 8 P-cores (using one logical core per physical P-core) or 8 E-cores, and the client threads on a fixed and separate set of 8 E-cores for all experiments.

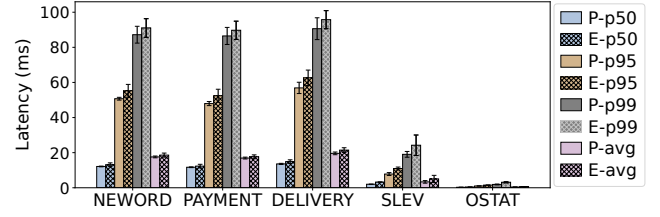


Figure 5: TPROC-C latency for five transaction types with PostgreSQL server threads on P- and E-cores.

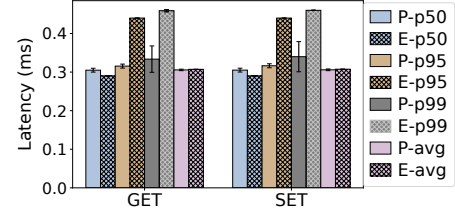


Figure 6: Memtier latency for GET and SET transactions with memcached server threads on P- and E-cores.

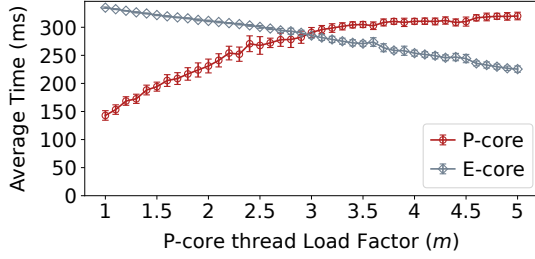
E-cores increase the percentile and average latencies for all the five transaction types in TPROC-C (Figure 5). For the DELIVERY type, which has the highest average latency, E-cores increase the average (over multiple runs) 50th, 90th, 95th, and average transaction latency by 9.3%, 10.1%, 5.7%, and 9.5% respectively over P-cores. The impact is less than the core frequency ratios due to overheads such as from accesses to the cache hierarchy and main memory. E-cores reduce the average transaction throughput, as measured by the NOPM metric [35], by 4.8% from that with 8 P-cores, and is close to the throughput with 4 P-cores. The tail latency differences are more pronounced for memtier (Figure 6) with its sub-millisecond latencies. Compared to P-cores, E-cores increase the 90th and 95th percentile latencies for GET transactions by 39.5% and 37.6% respectively and by 39.1% and 35.3% for SET transactions. However, machine power used is substantially lower (~ 169 W) with E-cores running memcached instead of P-cores (~ 231 – 239 W).

Thus, like for analytical workloads, running transactional workloads on E-cores instead of on P-cores saves significant power. However, that may considerably increase short tail latencies. With sufficient E-cores, idle cores might be used to improve single-thread latency through techniques such as speculative execution [7, 47], inter-core prefetching [23], and dataflow execution [17].

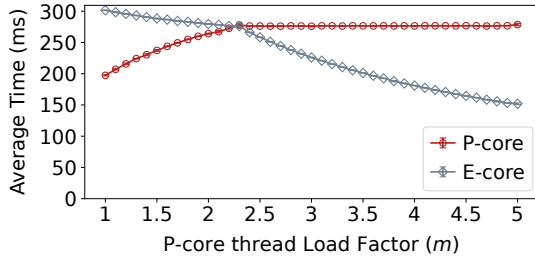
4 WORK DISTRIBUTION FOR HYBRID CORES

Next, we investigate if query processing, in particular work distribution, should be done any differently on platforms with hybrid cores. Our workload is grouped aggregation of a billion rows with a grouping column of integers and an aggregation column of data type double. We use 16 E-cores, and a variable number, n_p , of P-core threads. Each P-core thread processes m times the number of rows processed by each E-core. We call m the P-core thread load factor, with $m \neq 1$ reflecting non-uniform work distribution. We use robin hood hashing [14] similar to prior work [11], but we use its default

hash function and do the final merge of the per-thread aggregate results with a single thread.



(a) 4 P-core threads, 16 E-cores



(b) 16 P-core threads, 16 E-cores

Figure 7: Avg. thread times for aggregation with 1024 groups.

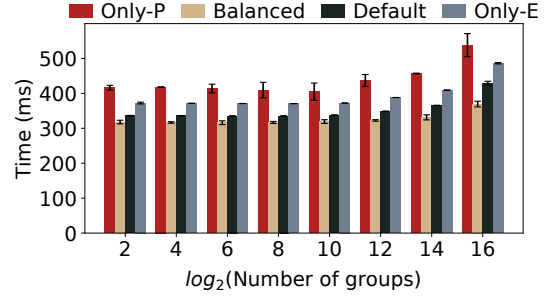
Figure 7 shows average P-core and E-core aggregation times for 1024 groups with $n_p = 4$ (on 4 physical P-cores) and with $n_p = 16$ (= 8 physical P-cores on this CPU). As m increases, average E-core time reduces due to less work while average P-core time rises. Let m_c denote the m corresponding to the crossover point for the two series. $m_c = 3$ and 2.3 for $n_p = 4$ and 16 respectively. For $m > m_c$, average P-core time saturates for $n_p = 16$ in spite of load shifted from E-cores, due to the limited memory bandwidth (~ 57 GB/sec) on this system being the critical bottleneck for this case.

A work allocation policy that aims to (non-uniformly) distribute work so that m is at or near m_c will balance the average P-core and E-core times. We call such a policy Balanced. Figure 8 shows the workload time that could be achieved with the following policies.

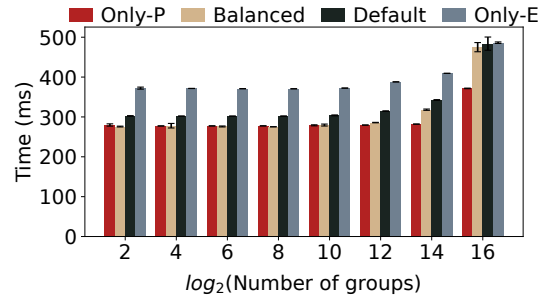
- Only-P: Uses only n_p P-core threads.
- Balanced: Uses n_p P-core threads and 16 E-cores, with m equal to (or close to) m_c . We find m_c and run time for Balanced through a post-processing step after running the workload with multiple values for $m \in [1, 5]$.
- Default: Uses n_p P-core threads and 16 E-cores, with $m = 1$.
- Only-E: Uses only 16 E-cores.

Apart from Default, the three other policies distribute work non-uniformly between P- and E-cores.

For $n_p = 4$, Balanced is always the best-performing policy with a speedup of 5%–16% over Default, depending on the number of groups (g). Only-P is the worst policy followed by Only-E. However, for $n_p = 16$, Only-P and Balanced achieve speedups of 8%–30% and 2%–10% respectively over Default, depending on g . Balanced is close to Only-P for $g \leq 4096$ (i.e., $\log_2(g) \leq 12$), when all per-thread hash tables fit within the L2 caches, but loses out for $g \geq$



(a) 4 P-core threads, 16 E-cores



(b) 16 P-core threads, 16 E-cores

Figure 8: Avg. workload time for different group counts.

16384 (i.e., $\log_2(g) \geq 14$) due to increased cache contention and L3 cache misses with the larger per-thread hash tables needed. With no E-cores active for Only-P, pressure on the L3 cache is reduced, while having enough parallelism and computational speed with the 16 faster P-core threads, leading to good performance for Only-P. The cache performance could potentially be improved with a more compact hash table implementation. Only-E performs the worst for all group sizes considered in this study.

We conclude that asymmetric work distribution policies that skew towards faster cores but adjust the load factor based on the workload parameters and system resources can be advantageous for hybrid systems.

5 CONCLUSION AND FUTURE WORK

Hybrid/asymmetric multicore CPUs can offer good performance for both single-threaded and well-parallelized multi-threaded workloads under area, power, and thermal budget constraints. How much resources to allocate for each core type depends on the optimization goal of performance or efficiency and the expected workload mix.

In this paper, we studied how database workloads perform on a recent hybrid CPU with P-cores and E-cores, estimated a minimum E-core:P-core ratio for equivalent performance of an analytical workload, and identified the potential to improve grouped aggregation performance. Future work can include power-performance studies on E-core only servers, workload characterization on hybrid CPUs with additional workload types, e.g., those including both relational and machine learning operations, and building a runtime scheduler for hybrid CPUs to speedup query processing with asymmetric work distributions.

REFERENCES

- [1] Mert Akdere, Ugur Çetintemel, Matteo Riondato, Eli Upfal, and Stanley B. Zdonik. 2012. Learning-based Query Performance Modeling and Prediction. In *2012 IEEE 28th International Conference on Data Engineering*. 390–401. <https://doi.org/10.1109/ICDE.2012.64>
- [2] AMD. 2024 (last accessed). AMD Ryzen Technology: Precision Boost 2 Performance Enhancement. <https://www.amd.com/en/resources/support-articles/faqs/CPU-PB2.html>.
- [3] Gene M. Amdahl. 1967. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference* (Atlantic City, New Jersey) (AFIPS '67 (Spring)). Association for Computing Machinery, New York, NY, USA, 483–485. <https://doi.org/10.1145/1465482.1465560>
- [4] Murali Annavaram, Ed Grochowski, and John Shen. 2005. Mitigating Amdahl's Law through EPI Throttling. In *Proceedings of the 32nd Annual International Symposium on Computer Architecture (ISCA '05)*. IEEE Computer Society, USA, 298–309. <https://doi.org/10.1109/ISCA.2005.36>
- [5] ARM. 2024 (last accessed). ARM DynamIQ. <https://www.arm.com/technologies/dynamiq>.
- [6] ARM. 2024 (last accessed). Processing Architecture for Power Efficiency and Performance. <https://www.arm.com/technologies/big-little>.
- [7] Azer Bestavros. 1993. Speculative Concurrency Control for Real-Time Databases. <https://open.bu.edu/handle/2144/1480>
- [8] Gavin Bonshor. 2024 (last accessed). Intel Previews Sierra Forest with 288 E-Cores, Announces Granite Rapids-D for 2025 Launch at MWC 2024. <https://www.anandtech.com/show/21276/intel-previews-sierra-forest-with-288-e-cores-announces-granite-rapids-d-for-2025-launch-at-mwc-2024>.
- [9] Alexander J. Branover, Benjamin Tsien, and Elliot H. Mednick. 2021. Method of task transition between heterogenous processors. *Patent Application Publication* US 20210173715A1 (2021).
- [10] Matthew Connatser. 2023. AMD is making a hybrid CPU, and here's what it will look like. <https://www.xda-developers.com/amd-hybrid-cpu-what-it-will-look-like/>.
- [11] Zubeyr F Eryilmaz, Aarati Kakaraparthi, Jignesh M Patel, Rathijit Sen, and Kwanghyun Park. 2021. FPGA for aggregate processing: The good, the bad, and the ugly. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1044–1055.
- [12] Zhiwei Fan, Rathijit Sen, Paraschos Koutris, and Aws Albarghouthi. 2020. Automated tuning of query degree of parallelism via machine learning. In *Proceedings of the Third International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. 1–4.
- [13] Archana Ganapathi, Harumi Kuno, Umeshwar Dayal, Janet L. Wiener, Armando Fox, Michael Jordan, and David Patterson. 2009. Predicting Multiple Metrics for Queries: Better Decisions Enabled by Machine Learning. In *2009 IEEE 25th International Conference on Data Engineering*. 592–603. <https://doi.org/10.1109/ICDE.2009.130>
- [14] Thibaut Goetghebuer-Planchon et al. 2024 (last accessed). A C++ implementation of a fast hash map and hash set using robin hood hashing. <https://github.com/Tessil/robin-map/>.
- [15] Ed Grochowski, Ronny Ronen, John Shen, and Hong Wang. 2004. Best of Both Latency and Throughput. In *Proceedings of the IEEE International Conference on Computer Design (ICCD '04)*. IEEE Computer Society, USA, 236–243.
- [16] The PostgreSQL Global Development Group. 2024 (last accessed). PostgreSQL: The World's Most Advanced Open Source Relational Database. <https://www.postgresql.org/>.
- [17] Gagan Gupta and Gurindar S. Sohi. 2011. Dataflow execution of sequential imperative programs on multicore architectures. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture* (Porto Alegre, Brazil) (MICRO-44). Association for Computing Machinery, New York, NY, USA, 59–70. <https://doi.org/10.1145/2155620.2155628>
- [18] Mark D. Hill and Michael R. Marty. 2008. Amdahl's Law in the Multicore Era. *Computer* 41, 7 (2008), 33–38. <https://doi.org/10.1109/MC.2008.209>
- [19] Intel. 2024 (last accessed). How Intel Technologies Boost Your CPU's Performance. <https://www.intel.com/content/www/us/en/gaming/resources/how-intel-technologies-boost-cpu-performance.html>.
- [20] Intel. 2024 (last accessed). Information about Temperature for Intel Processors. <https://www.intel.com/content/www/us/en/support/articles/000005597/processors.html>.
- [21] Intel. 2024 (last accessed). Intel Core i9 Processor 14900K. <https://ark.intel.com/content/www/us/en/ark/products/236773/intel-core-i9-processor-14900k-36m-cache-up-to-6-00-ghz.html>.
- [22] Intel. 2024 (last accessed). Intel Performance Counter Monitor (Intel PCM). <https://github.com/intel/pcm>.
- [23] Md Kamruzzaman, Steven Swanson, and Dean M. Tullsen. 2011. Inter-core prefetching for multicore processors using migrating helper threads. In *Proceedings of the Sixteenth International Conference on Architectural Support for Programming Languages and Operating Systems* (Newport Beach, California, USA) (ASPLOS XVI). Association for Computing Machinery, New York, NY, USA, 393–404. <https://doi.org/10.1145/1950365.1950411>
- [24] Stefanos Kaxiras and Margaret Martonosi. 2008. *Computer Architecture Techniques for Power-Efficiency* (1st ed.). Morgan and Claypool Publishers.
- [25] Rakesh Kumar, Dean M. Tullsen, Norman P. Jouppi, and Parthasarathy Ranganathan. 2005. Heterogeneous Chip Multiprocessors. *Computer* 38, 11 (nov 2005), 32–38. <https://doi.org/10.1109/MC.2005.379>
- [26] Dumitrel Loghin, Bogdan Marius Tudor, Hao Zhang, Beng Chin Ooi, and Yong Meng Teo. 2015. A performance study of Big Data on small nodes. *PVLDB* 8, 7 (2015), 762–773.
- [27] Ryan Marcus and Olga Papaemmanouil. 2019. Plan-structured deep neural network models for query performance prediction. *PVLDB* 12, 11 (jul 2019), 1733–1746. <https://doi.org/10.14778/3342263.3342646>
- [28] Tomer Y. Morad, Uri C. Weiser, Avinoam Kolodny, Mateo Valero, and Eduard Ayguade. 2006. Performance, Power Efficiency and Scalability of Asymmetric Cluster Chip Multiprocessors. *IEEE Computer Architecture Letters* 5, 1 (jan 2006), 4–17. <https://doi.org/10.1109/L-CA.2006.6>
- [29] Danica Porobic, Ippokratis Pandis, Miguel Branco, Pinar Tözün, and Anastasia Ailamaki. 2012. OLTP on hardware islands. *PVLDB* 5, 11 (jul 2012), 1447–1458. <https://doi.org/10.14778/2350229.2350260>
- [30] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) (SIGMOD '19). Association for Computing Machinery, New York, NY, USA, 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [31] Redis. 2024 (last accessed). memtier_benchmark. https://github.com/RedisLabs/memtier_benchmark.
- [32] Nikhil Rukmabhatla, Rajshree Chabukwar, Sneha Gohad, and Michael Chynoweth. 2024 (last accessed). Intel performance hybrid architecture & software optimizations. https://cdrdv2-public.intel.com/685865/211112_Hybrid_WP_2_Developing_v1.2.pdf.
- [33] Rathijit Sen and Karthik Ramachandra. 2018. Characterizing resource sensitivity of database workloads. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 657–669. <https://doi.org/10.1109/HPCA.2018.00062>
- [34] Rathijit Sen and Yuanyuan Tian. 2023. Microarchitectural Analysis of Graph BI Queries on RDBMS. In *Proceedings of the 19th International Workshop on Data Management on New Hardware*. 102–106.
- [35] Steve Shaw et al. 2024 (last accessed). Comparing HammerDB results. <https://www.hammerdb.com/docs/ch03s04.html>.
- [36] Steve Shaw et al. 2024 (last accessed). HammerDB. <https://github.com/TPC-Council/HammerDB>.
- [37] Steve Shaw et al. 2024 (last accessed). HammerDB TPROC-C workload. <https://www.hammerdb.com/docs/ch03s03.html>.
- [38] Utku Sirin. 2021. *Micro-architectural Analysis of Database Workloads*. Ph.D. Dissertation. EPFL. <https://doi.org/10.5075/epfl-thesis-10363>
- [39] Utku Sirin and Anastasia Ailamaki. 2020. Micro-Architectural Analysis of OLAP: Limitations and Opportunities. *PVLDB* (2020), 840–853. <https://doi.org/10.14778/3380750.3380755>
- [40] Utku Sirin, Sandhya Dwarkadas, and Anastasia Ailamaki. 2021. Performance Characterization of HTAP Workloads. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 1829–1834. <https://doi.org/10.1109/ICDE51399.2021.00162>
- [41] Utku Sirin, Pinar Tözün, Danica Porobic, Ahmad Yasin, and Anastasia Ailamaki. 2021. Micro-Architectural Analysis of in-Memory OLTP: Revisited. *The VLDB Journal* 30, 4 (mar 2021), 641–665. <https://doi.org/10.1007/s00778-021-00663-8>
- [42] Pinar Tözün and Helena Kotthaus. 2019. Scheduling data-intensive tasks on heterogeneous many cores. *IEEE Data Engineering Bulletin* 42, 1 (2019), 61–72.
- [43] Wikipedia. 2024 (last accessed). Alder Lake. https://en.wikipedia.org/wiki/Alder_Lake.
- [44] Wikipedia. 2024 (last accessed). Gracemont (microarchitecture). [https://en.wikipedia.org/wiki/Gracemont_\(microarchitecture\)](https://en.wikipedia.org/wiki/Gracemont_(microarchitecture)).
- [45] Wikipedia. 2024 (last accessed). Raptor Cove. https://en.wikipedia.org/wiki/Golden_Cove#Raptor_Cove.
- [46] Wikipedia. 2024 (last accessed). Raptor Lake. https://en.wikipedia.org/wiki/Raptor_Lake.
- [47] Antonia Zhai, Christopher B Colohan, J Gregory Steffan, and Todd C Mowry. 2002. Compiler optimization of scalar value communication between speculative threads. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*. 171–183.